

DEVOPS FUNDAMENTALS | FINAL PROJECT

Project Brief

Containerised Microservice with Automated CI/CD and Cloud Deployment

Marks: 10 | Weightage: 10% | Duration: 3.5 Weeks | Individual Project

IMPORTANT INSTRUCTIONS

1. This is an individual project. Any form of plagiarism or code sharing will result in zero marks for all involved students.
2. Submission is a public GitHub repository link + live EC2 URL + a 5-minute demo video (screen recording).
3. Your registration number must appear in three places: /health endpoint response, README header, and Git author email.
4. The EC2 instance must be running at the time of evaluation. A dead server = zero marks for AWS component.

1. Project Overview

What you are building and why

You will build and deploy a production-ready containerised microservice application that demonstrates your mastery of the complete DevOps toolchain taught in this course. This is not a software development project — the application code is provided to you as a starter template. Your job is entirely focused on the DevOps infrastructure: containerisation, CI/CD pipeline, automated testing, and cloud deployment.

By the end of this project you will have built something real — a live application running on a cloud server that automatically tests and deploys itself every time you push code to GitHub. This is exactly how professional DevOps engineers work at companies around the world.

Tools You Will Use

Linux — Server setup, file management, shell scripting, SSH into EC2

Git & GitHub — Feature branches, pull requests, commit history, repository management

Docker — Dockerfile, image building, container management, .dockerignore

Docker Compose — Multi-container orchestration, volumes, health checks, environment variables

GitHub Actions — CI pipeline (lint + test), CD pipeline (auto-deploy to EC2 on push)

AWS EC2 — Free tier Ubuntu instance, Security Groups, SSH key management

PostgreSQL — Database container, persistent named volume, DB connection from app

Submit all of the following by the deadline. Incomplete submissions will be marked on what is working only.

- GitHub repository URL (must be public)
- Live EC2 public IP address with port — e.g. `http://3.92.145.12:8000`
- Screenshot of green CI/CD pipeline from GitHub Actions tab
- 5-minute screen recording demo showing: app running on EC2, POST a student record, GET to confirm it persisted, GitHub Actions green pipeline
- README.md must include: your name, registration number, setup instructions, and architecture description

2. What You Are Building

Application architecture and requirements

A FastAPI web application with a PostgreSQL database backend. The app exposes four REST API endpoints and persists data in a PostgreSQL container. Both services run via Docker Compose. The entire system deploys automatically to AWS EC2 whenever you push code to the main branch.

2.1 Application Architecture

Component	Technology	Description
Web Service	FastAPI + Uvicorn	REST API running on port 8000
Database	PostgreSQL 15	Relational DB on port 5432 with named volume
CI Pipeline	GitHub Actions	Runs flake8 + pytest on every push and PR
CD Pipeline	GitHub Actions + SSH	Auto-deploys to EC2 on every push to main
Cloud Server	AWS EC2 t2.micro (free tier)	Ubuntu server running Docker + Compose

2.2 API Endpoints

Your application must implement all four of these endpoints exactly:

Method	Endpoint	Expected Response
GET	/health	{ "status": "ok", "db": "connected", "student": "YOUR_REG_NO" }
POST	/students	Accepts JSON body, saves to DB, returns created record with ID
GET	/students	Returns list of all student records from PostgreSQL
GET	/students/{reg_no}	Returns single student record or 404 if not found

2.3 Required Project Structure

Your GitHub repository must follow this exact folder structure:

```
YOUR_REGNUM-devops-project/
├── app/
│   ├── main.py          ← FastAPI routes
│   ├── database.py      ← SQLAlchemy DB setup
│   ├── models.py        ← Student data model
│   └── tests/
│       ├── conftest.py   ← pytest fixtures and test DB
│       ├── test_health.py ← Tests for /health
│       └── test_students.py ← Tests for POST/GET /students
├── Dockerfile
├── docker-compose.yml    ← Local development
├── docker-compose.prod.yml ← Production (EC2)
├── requirements.txt
├── .env.example          ← Template only — never commit real .env
├── .gitignore
├── .dockerignore
├── .github/
│   └── workflows/
│       ├── ci.yml        ← Lint + test
│       └── cd.yml        ← Deploy to EC2
└── README.md            ← Setup guide + architecture
```

3. Marking Breakdown

How your 10 marks are awarded

Component	Marks	What the examiner checks
1. Linux & Git Workflow	1.5	10+ meaningful commits, 2+ feature branches merged via PR, .gitignore correct, README complete
2. Docker & Dockerfile	1.5	Slim base image, correct layer order, non-root user, .dockerignore, builds without errors
3. Docker Compose	2.0	Two services, named volume, health check + depends_on, .env variables, app connects to DB
4. GitHub Actions CI	2.0	Triggers on push + PR, flake8 passes, 3+ pytest tests pass, test DB used in pipeline
5. AWS EC2 + CD Pipeline	3.0	EC2 running, GitHub Secrets used, auto-deploy on push, live URL accessible, data persists
TOTAL	10	10% of Final Grade

Submission Checklist

- [] GitHub repo is public and named: YOUR_REGNUM-devops-project
- [] Registration number appears in /health response
- [] At least 10 commits with meaningful messages (feat:, fix:, docs:)
- [] At least 2 feature branches merged via Pull Request
- [] docker compose up works locally — both services start
- [] GitHub Actions CI: flake8 + pytest all green
- [] GitHub Actions CD: triggers on push to main, SSHs into EC2, redeploys
- [] EC2 instance is running and app is accessible on public IP:8000
- [] POST /students → data appears in GET /students (persistence confirmed)
- [] Demo video submitted (5 minutes, screen recording)